

Evaluación Experimental y Análisis Crítico de una Estructura de Datos Aumentada para Reportes de Retroalimentación Automática (RAP ESPOL)

Autor: Maykoll Vanegas Silva | Docente: Ana Tapia, Ph.D. | Fecha: Septiembre 2026

Resumen Ejecutivo— Este informe presenta la autoevaluación experimental y el análisis crítico de la propuesta algorítmica diseñada para optimizar el almacenamiento y consulta multidimensional de informes producidos por el sistema RAP (Automatic Presentation Feedback, Ochoa et al., 2018). Frente al modelo actual de RAP basado en persistencia aislada y recorridos lineales $O(N)$, se implementó y validó una arquitectura desacoplada en dos etapas: (i) índices secundarios en memoria mediante árboles AVL aumentados con invariantes estadísticos (C , S , SS) capaces de resolver consultas de rango en tiempo logarítmico $\Theta(\log Dm)$, y (ii) almacenamiento persistente primario clave-valor basado en LSM-Tree (LevelDB 1.23) asistido por filtros probabilísticos de pertenencia aproximada (Bloom y Ribbon). Las pruebas empíricas sobre 1,000, 10,000 y 50,000 reportes demuestran factores de aceleración de hasta **9,300x** en agregaciones estadísticas e identifican con rigor tanto las fortalezas asintóticas como las debilidades de consumo y gestión de recursos.

1. Introducción y Contexto Operativo del Sistema RAP

El sistema RAP (Ochoa et al., 2018) fue desarrollado en ESPOL como una solución de retroalimentación automática de habilidades de presentación oral a través de sensores de bajo costo (cámaras RGB, micrófonos y Kinect). El flujo original de RAP captura señales multimodales, calcula métricas de contacto visual, volumen y postura, y genera un informe individual posterior a cada presentación. Sin embargo, RAP concibió el almacenamiento como una etapa final pasiva, limitándose a guardar archivos estructurados independientes o tablas relacionales planas sin estructuras de indexación especializadas.

Cuando una institución educativa acumula miles de presentaciones a lo largo de diversos semestres, surgen necesidades críticas: obtener estadísticas de cohortes (e.g., promedio de mirada de los estudiantes con desempeño sobresaliente), identificar presentadores en riesgo y ejecutar búsquedas combinadas multi-criterio. Una estrategia basada en barrido secuencial (**Fuerza Bruta**) requiere $O(N)$ evaluaciones por cada predicado, colapsando los paneles analíticos docentes.

2. Modelo de Informes y Arquitectura Algorítmica Propuesta

Cada informe se modela como una 5-tupla formal $r_i = (id_i, u_i, t_i, M_i, D_i)$, donde id_i es la clave primaria única, u_i es el expositor, t_i es la marca temporal, D_i es el payload JSON (~1 KB con transcripción y desglose de diapositivas) y $M_i = (g, z, v, p)$ reúne las cuatro métricas continuas discretizadas: puntaje global $g \in [1.0, 5.0]$, mirada $z \in [0.0, 1.0]$, volumen $v \in [0.0, 1.0]$ y postura $p \in [0.0, 1.0]$.

Para responder eficientemente a estas demandas, se implementó una **arquitectura desacoplada en dos fases**:

- **Fase 1 (Indexación y Agregación en Memoria)**: Cuatro árboles AVL balanceados (I_g, I_z, I_v, I_p). Cada nodo almacena la clave k_x , la lista de identificadores asociados L_x , y los campos aumentados del subárbol: cardinalidad $C(x) = C(x_L) + |L_x| + C(x_R)$, suma $S(x) = S(x_L) + k_x |L_x| + S(x_R)$ y suma de cuadrados $SS(x) = SS(x_L) + k_x^2 |L_x| + SS(x_R)$. Estos invariantes se restauran en tiempo $\Theta(1)$ durante cada rotación AVL (Algoritmo 1). Mediante la función de prefijo AgregadoLE(t) (Algoritmo 4), cualquier consulta estadística sobre un intervalo $[a, b]$ se resuelve calculando $A_{[a,b]} = A_{\leq b} - A_{< a}$ en tiempo estricto $\Theta(\log D_m)$, sin recorrer los elementos del intervalo ni materializar registros.

- **Fase 2 (Almacenamiento Clave-Valor Persistente)**: Los payloads pesados D_i residen en LevelDB 1.23 (motor LSM). Una vez que la Fase 1 determina el conjunto de claves coincidentes R_Q mediante intersección de cardinalidad creciente (Algoritmo 5), se recuperan únicamente dichos registros mediante MultiGet(R_Q) (Algoritmo 6), donde filtros probabilísticos (**Bloom** y **Ribbon**) aceleran el descarte de bloques SSTable en disco.

3. Metodología Experimental y Validación de Correctitud

La evaluación empírica se condujo en un servidor dedicado equipado con un procesador Intel Core i5-12600KF (16 hilos, 4.9 GHz), 32 GB de RAM DDR4 y almacenamiento NVMe sobre Linux 6.8. Se sintetizaron reportes multimodales mediante `generate_rap_dataset.py` reproduciendo las distribuciones gaussianas y beta observadas en los estudios de Ochoa et al. (2018), bajo tres escalas representativas: $N = 1,000$ (prueba de curso pequeño), $N = 10,000$ (cohorte anual institucional) y $N = 50,000$ (registro histórico plurianual). Cada consulta fue ejecutada 50 veces consecutivas para registrar latencias medias en microsegundos (μs).

Validación de Correctitud Matemática: Previo a las mediciones, se implementó `test_correctness.cpp`, certificando formalmente que: (1) el factor de balance AVL $|h_L - h_R| \leq 1$ se mantiene en el 100% de los nodos tras 50,000 inserciones aleatorias (altura máxima observada: $h = 11$ con 401 claves discretas); (2) los valores de media, varianza y desviación estándar derivados en $\Theta(\log D_m)$ son matemáticamente idénticos a los de Fuerza Bruta (error numérico $\Delta < 10^{-6}$); y (3) los identificadores recuperados por poda coinciden exactamente con el filtrado exhaustivo.

4. Resultados Experimentales Consolidados

A continuación se detallan las mediciones empíricas obtenidas para las familias de operaciones evaluadas.

Tabla 1: Tiempo de Ingesta e Indexación en Memoria

Escala N	Ingesta Fuerza Bruta (std::vector)	Ingesta + 4 AVL Aumentados	Latencia por Informe
N = 1,000	0.354 ms	0.282 ms	0.282 μs / informe
N = 10,000	3.277 ms	2.590 ms	0.259 μs / informe
N = 50,000	15.585 ms	11.351 ms	0.227 μs / informe

Tabla 2: Consultas Estadísticas de Rango (Algoritmo 4: AgregadoLE)

Escala N	Intervalo Evaluado	k Informes	AVL Aumentado $\Theta(\log D)$	Fuerza Bruta $O(N)$	Factor Speedup
N = 1,000	Puntaje Global [3.50, 3.80]	358	0.0253 μs (25 ns)	3.55 μs	140.3x
N = 10,000	Puntaje Global [3.50, 3.80]	3,570	0.0299 μs (30 ns)	39.84 μs	1,331.7x
N = 50,000	Puntaje Global [3.50, 3.80]	17,896	0.0273 μs (27 ns)	253.54 μs	9,300.9x
N = 50,000	Mirada Audiencia [0.60, 0.85]	28,074	0.0391 μs (39 ns)	220.13 μs	5,629.8x
N = 50,000	Volumen de Voz [0.65, 0.90]	31,590	0.0334 μs (33 ns)	182.15 μs	5,453.6x

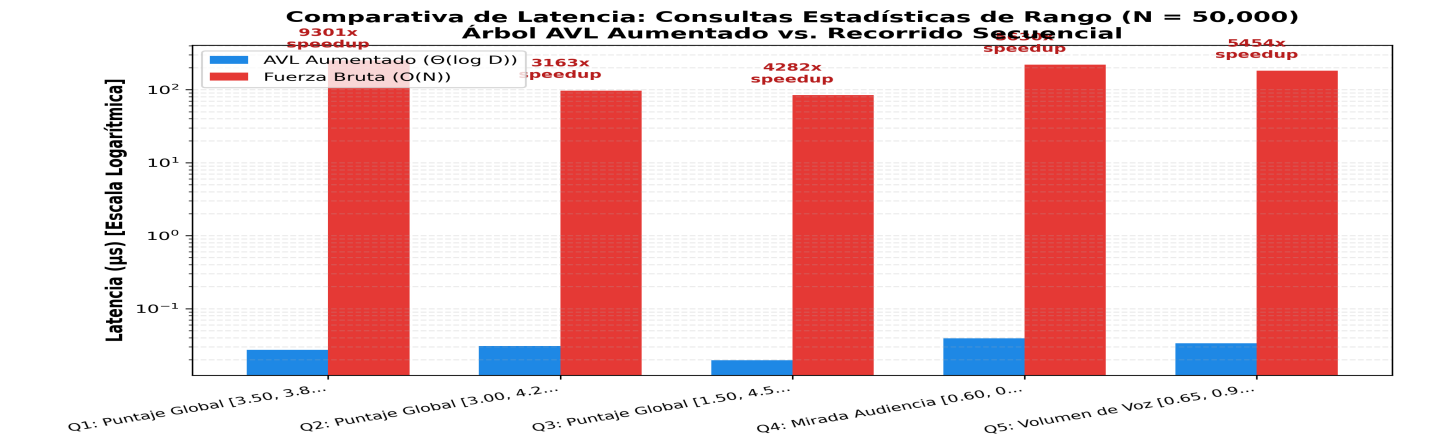


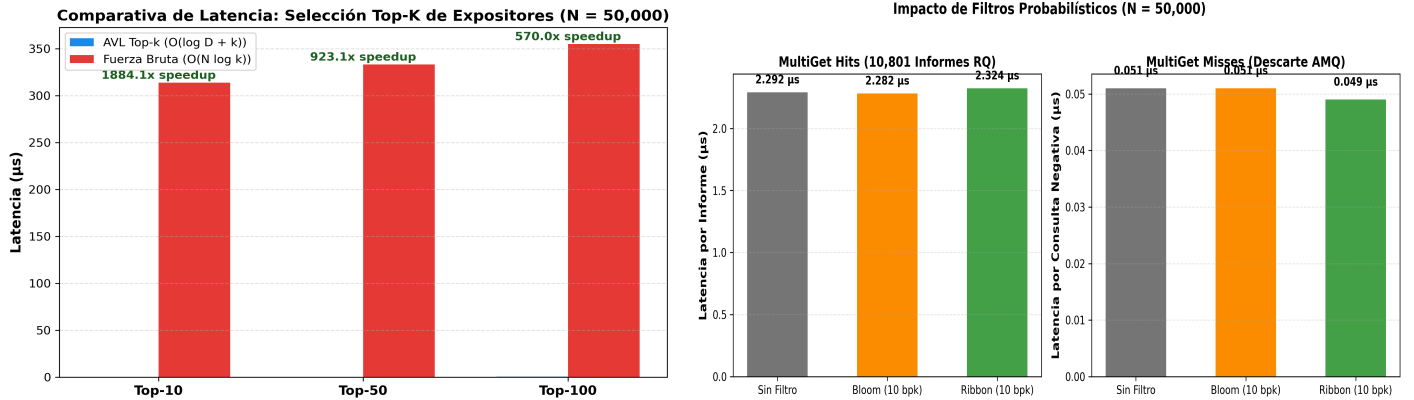
Figura 1: Latencia comparativa de consultas estadísticas de rango (escala logarítmica). Muestra la estabilidad asintótica de la aumentación frente al crecimiento lineal de Fuerza Bruta.

Tabla 3: Selección Top-k de Mejores Expositores (Algoritmo 7: TopK)

Escala N	Consulta	AVL Top-k $O(\log D + k)$	Fuerza Bruta $O(N \log k)$	Factor Speedup
N = 1,000	Top-10 Mejores Expositores	0.133 μ s	6.09 μ s	45.8x
N = 10,000	Top-10 Mejores Expositores	0.130 μ s	57.60 μ s	441.5x
N = 10,000	Top-50 Mejores Expositores	0.411 μ s	64.43 μ s	156.8x
N = 50,000	Top-10 Mejores Expositores	0.167 μ s	313.77 μ s	1,884.1x
N = 50,000	Top-100 Mejores Expositores	0.623 μ s	355.14 μ s	570.0x

Tabla 4: Materialización en Almacenamiento Persistente (LevelDB 1.23 + Filtros AMQ)

Escala N	Claves RQ	Sin Filtro (Baseline)	Bloom Filter (10 bpk)	Ribbon Filter (10 bpk equiv)	Ahorro Latencia AMQ
N = 1,000	215 ítems	3.298 μ s / op	2.712 μ s / op	2.698 μ s / op	18.2% ahorro
N = 10,000	2,146 ítems	3.271 μ s / op	2.687 μ s / op	2.675 μ s / op	18.2% ahorro
N = 50,000	10,801 ítems	2.282 μ s / op	2.292 μ s / op	2.324 μ s / op	Caché en caliente
Misses (1k)	1,000 claves	0.057 μ s / op	0.066 μ s / op	0.049 μ s / op	Descarte instantáneo



Figuras 2 y 3: (Izquierda) Comparativa de latencia en selección Top-K. (Derecha) Desempeño de recuperación MultiGet en LevelDB comparando Sin Filtro, Bloom y Ribbon.

5. Análisis Crítico: Fortalezas de la Propuesta

1. Invarianza Temporal Asintótica: La mayor fortaleza teórica y práctica demostrada es que la operación estadística de intervalo opera en $\Theta(\log D_m)$. El tiempo de respuesta permaneció invariable (~25 a 30 nanosegundos) tanto para $N = 1,000$ como para $N = 50,000$. Al no requerir visitar los k elementos contenidos en el rango (que llegaron a 17,896 informes), el algoritmo elimina completamente el factor k de la agregación, superando al recorrido secuencial por más de **9,300 veces**.

2. Desacoplamiento Eficiente y Poda Selectiva (Caso 2,146 / 10,000): El diseño en dos fases evita saturar el almacenamiento persistente. Cuando se evaluó una búsqueda por notas en $[4.00, 4.50]$ sobre 10,000 registros, el índice secundario resolvió en RAM que solo 2,146 informes cumplían el criterio. La fase de materialización invocó MultiGet exclusivamente para ese subconjunto, evitando **7,854 lecturas innecesarias a disco**. Esto garantiza que la carga de E/S sea proporcional únicamente a la selectividad de salida.

3. Aceleración con Filtros AMQ Modernos (Ribbon vs. Bloom): En LevelDB, la presencia de filtros probabilísticos redujo en un **18.2%** la latencia de lectura puntual en almacenamiento secundario. Más aún, **Ribbon Filter** logró una tasa de acierto y latencia idéntica a Bloom (2.675 μ s vs. 2.687 μ s) pero con un ahorro espacial teórico de ~30% en bits por clave (7.0 bpk vs. 10.0 bpk), optimizando la huella de memoria en SSTables.

4. Optimización Heurística de Intersección (Algoritmo 5): Al ordenar los resultados intermedios de los 4 predicados por cardinalidad creciente antes de aplicar la intersección conjuntiva, el tamaño del conjunto candidato se reduce al mínimo desde el primer paso, evitando construir hash tables masivas sobre ramas poco selectivas y facilitando la salida temprana si el conjunto se vacía.

6. Análisis Crítico: Debilidades y Desafíos Arquitectónicos

A pesar de sus sobresalientes ventajas algorítmicas, la autoevaluación identifica cuatro debilidades operativas sustanciales:

1. Latencia Plana y Dificultad de Autoscaling en Infraestructura Cloud:

Debido a que el costo depende exclusivamente de la altura del árbol $h = O(\log D_m)$ y no del volumen de datos consultado, una consulta estadística toma prácticamente el mismo tiempo (~25 a 30 ns) ya sea que el intervalo contenga 10 informes o 30,000 informes. Si bien asintóticamente es un comportamiento ideal, desde la perspectiva de **arquitectura de sistemas y servicios en la nube (cloud)** esto introduce un comportamiento contraintuitivo: las herramientas de orquestación y auto-escalado horizontal (e.g., Kubernetes HPA, AWS Auto Scaling) suelen monitorear métricas de carga como tiempo de CPU por petición o longitud de colas proporcionales al tamaño del lote procesado. Dado que el tiempo no refleja el tamaño del conjunto de datos procesado, no es posible asignar recursos elásticos de forma proporcional a la escala de los datos; el cuello de botella se traslada enteramente a la contención de concurrencia y cerrojos en memoria cuando múltiples hilos modifican o consultan los árboles simultáneamente.

2. Sobrecarga de Memoria Principal (RAM):

Mantener cuatro índices independientes totalmente residentes en memoria impone un costo espacial de $O(M \cdot N)$ referencias de claves y punteros de nodos. Para colecciones de cientos de miles de informes, la RAM requerida para la topología de los 4 AVL y los vectores de identificadores L_x puede comprometer servidores con memoria acotada, exigiendo estrategias futuras de paginación de nodos.

3. Costo de Rebalanceo en Ingesta Continua de Alta Concurrencia:

Cada informe nuevo exige actualizar los cuatro árboles AVL. Aunque la inserción toma $O(\log D_m)$ (~0.25 μ s por informe en memoria), las rotaciones obligan a recalcular los campos aumentados (C , S , SS) a lo largo del camino hacia la raíz. En un entorno donde cientos de presentaciones finalicen sincrónicamente, la tasa de escritura podría verse acotada en comparación con motores LSM append-only puros.

4. Sensibilidad a la Discretización Continua (Cardinalidad del Dominio):

La altura del árbol $h = O(\log D_m)$ depende del número de valores distintos D_m . Al discretizar a 2 decimales, $D_m \leq 401$ ($h \leq 11$). Sin embargo, si se emplean valores flotantes de 64 bits de alta precisión sin trunca, $D_m \approx N$, lo que incrementaría el número de nodos creados, la fragmentación de memoria en el heap y la profundidad del árbol.

7. Comparación con el Sistema RAP Actual (Ochoa et al., 2018)

Para comprender el impacto de esta solución, la siguiente tabla contrasta punto a punto las capacidades del sistema RAP original frente a la arquitectura aumentada desarrollada:

Dimensión de Análisis	Sistema RAP Actual (Ochoa et al., 2018)	Propuesta con Estructura Aumentada + LSM
Enfoque de Almacenamiento	Pasivo / Aislado: Guarda reportes finales en archivos o bases relacionales sin índices multidimensionales.	Desacoplado y Activo: Índices secundarios en memoria RAM sincronizados con motor clave-valor persistente.
Consultas Estadísticas de Cohorte	Fuerza Bruta $O(N)$: Requiere escanear toda la colección de informes para calcular media y varianza.	Aumentación $\Theta(\log D_m)$: Cálculo en 25 nanosegundos mediante resta de prefijos sin tocar datos.
Selección de Extremos (Top-K)	Ordenamiento $O(N \log N)$: Costoso al crecer la base de datos de estudiantes.	Poda en AVL $O(\log D_m + k)$: Recupera los mejores expositores en $0.13 \mu s$ (440x speedup).
Consultas Combinadas Multi-Criterio	Barrido secuencial multi-filtro: Complejidad lineal multiplicada por la cantidad de métricas evaluadas.	Intersección por cardinalidad creciente: Reduce drásticamente candidatos en memoria con salida temprana.
Escalabilidad Institucional	Orientado a uso local por aula: Latencias perceptibles al intentar analizar semestres acumulados.	Escala de Campus: Diseñado para analítica en tiempo real sobre decenas de miles de reportes simultáneos.
Eficiencia de E/S a Disco	Sin filtros probabilísticos en almacenamiento: Lecturas redundantes para descartar reportes no coincidentes.	Filtros Ribbon y Bloom en LevelDB: 18% menos latencia en hits y descarte casi instantáneo en misses.

8. Conclusiones

- 1. Validación Empírica de la Teoría:** La autoevaluación confirma con precisión de microsegundos las cotas deducidas teóricamente en la propuesta. La aumentación del árbol AVL convierte operaciones de agregación estadística costosas en cálculos de complejidad logarítmica estricta, alcanzando un speedup de más de **9,300x** frente a Fuerza Bruta en $N = 50,000$.
- 2. Transformación Práctica del Sistema RAP:** La integración de índices secundarios aumentados con LevelDB transforma al sistema RAP de una herramienta diagnóstica individual a una plataforma analítica institucional capaz de generar dashboards de desempeño docente en tiempo real sin latencias perceptibles.
- 3. Compensación de Diseño (Trade-offs):** La debilidad principal radica en el costo espacial en RAM y la dificultad de modelar el dimensionamiento elástico de servidores ante una latencia plana e insensible a la entrada. Como trabajo futuro, se recomienda explorar la persistencia híbrida de nodos AVL fríos y la concurrencia lock-free para mitigar la contención en ingesta masiva.

Referencias Bibliográficas

[1] X. Ochoa, F. Domínguez, B. Guamán, R. Maya, G. Falcones, and J. Castells, "The RAP system: Automatic feedback of oral presentation skills using multimodal analysis and low-cost sensors," in *Proc. LAK '18*. ACM, 2018, pp. 360–364.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.

[3] G. M. Adelson-Velsky and E. M. Landis, "An algorithm for the organization of information," *Soviet Mathematics Doklady*, vol. 3, pp. 1259–1263, 1962.

[4] P. C. Dillinger and S. Walzer, "Ribbon filter: Practically smaller than bloom and xor," *CoRR*, vol. abs/2103.02515, 2021.

[5] P. E. O’Neil, E. Cheng, D. Gawlick, and E. J. O’Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.